DetFuzz: A Semantic-Guided Fuzzing Framework via Operation Logic Inference for Black-box IoT Devices

Anonymous Author(s)

Abstract

The proliferation of Internet of Things (IoT) devices has greatly increased security vulnerabilities. Current vulnerability detection techniques typically rely on firmware or source code, limiting their applicability when such internal artifacts are inaccessible. Black-box fuzzing can operate in such scenarios but lacks reliable information to guide mutation. To address this limitation, we present DetFuzz, a novel fuzzing framework for IoT vulnerability detection with no reliance on firmware or source code. DetFuzz is motivated by a common observation in real-world black-box IoT scenarios: user-controllable inputs driving interface operations are more likely to reach sensitive sinks and often exhibit identifiable semantic characteristics. By leveraging these semantic characteristics to infer the actual relationships between operations and parameters in back-end code, it can address the lack of information that guides mutation in black-box fuzzing. However, identifying inputs to prioritize for mutation and inferring the prerequisites for reaching sensitive sinks remain challenging. Therefore, DetFuzz proposes Large Language Model (LLM)-based semantic mapping with similarity-based algorithms to identify inputs to prioritize for mutation. It further designs a prerequisite reasoning technique with additional front-end analysis information to address prerequisite bottlenecks.

To evaluate DetFuzz, we conducted extensive experiments on devices from 6 major IoT vendors. Results show that DetFuzz discovers over 7 times more vulnerabilities than the current state-of-the-art (SOTA) black-box and grey-box fuzzing methods, with 94.48% recall. Overall, DetFuzz detected 137 vulnerabilities, 122 of which were assigned CVE IDs. Among these, 40 were newly discovered only by DetFuzz and had an average CVSS 3.x score of 8.54, covering buffer overflow and command injection vulnerabilities. The source code is available at https://figshare.com/s/007bf99aca2252a63b75.

Keywords

Internet of Things, Fuzzing, Large Language Model

1 Introduction

In recent years, the number of active IoT devices has surpassed 20 billion and is projected to reach approximately 40 billion by 2030 [44, 45]. As IoT vendors increasingly adopt firmware encryption techniques and cloud-based management architectures, black-box testing scenarios for IoT devices have become increasingly prevalent [11, 19, 21, 24, 32, 34, 36, 38]. In black-box scenarios, the accessible information is limited to the device's web application, including its web front-end, intercepted network packets, and responses.

However, existing approaches for detecting IoT vulnerabilities struggle to operate effectively in black-box scenarios due to insufficient information. These approaches can be categorized into static and dynamic analysis. Static analysis methods, such as taint analysis and symbolic execution [22, 43, 49, 57, 58], detect vulnerabilities in IoT firmware or source code without execution, but are inapplicable in black-box scenarios where back-end binaries are inaccessible.

Dynamic analysis methods, represented by fuzz testing [21, 26–30, 33, 37, 42, 47, 56, 59], examine program behavior during execution and discover security flaws through techniques such as input mutation. They can be categorized into white-box, grey-box, and black-box methods. White-box and grey-box fuzzing typically require firmware and high-fidelity emulation [27, 28, 37, 47, 50, 59], and collect feedback, such as code coverage, to guide input generation, which also cannot be used in black-box settings. In contrast, black-box fuzzing tools, such as boofuzz [29] and snipuzz [26], can operate without internal information. However, this limitation also means they lack reliable information to guide their mutation strategies, reducing their detection effectiveness. For example, Snipuzz [26] uses device responses to guide mutations, but these responses are often too simplistic to provide reliable information for guidance. As current SOTA black-box solution, Labrador [30] uses response-driven control-flow inference. However, it relies on firmware for string extraction and cannot be applied to strict black-box settings.

To address vulnerability detection limitations in black-box scenarios, we conducted an empirical study (detailed in Appendix A) to investigate observable information that can guide input mutation and observed a prevalent phenomenon: On the one hand, user-controllable inputs driving interface operations are more likely to flow into sensitive sinks in back-end programs, such as memory operations or command executions, and therefore present higher security risks. On the other hand, developers tend to assign semantically meaningful names to user-controllable inputs. For instance, in CVE-2024-7175 [48], the `setDiagnosisCfg` interface is responsible for performing PING diagnostics on the device. In this operation, the user-provided "target address" input or parameter is named `ipDomain`, which is passed to `system()` as the PING destination without sanitization, resulting in a command injection vulnerability. Importantly, the interface `setDiagnosisCfg` and the parameter `ipDomain` are both semantically named, which could be helpful information to identify high-risk parameters and guide the fuzzing strategy. This observation motivates the core design of our methodology: leveraging semantic characteristics present in observable information to infer the actual relationships between operations and parameters in back-end code. By prioritizing fuzzing resources to the user-controllable inputs that drive operations, vulnerability discovery efficiency can be significantly improved.

However, turning these observations into effective fuzzing strategies poses two challenges: For prioritizing fuzzing resources to high-risk parameters, we need to identify the user-controllable inputs that drive operations and ensure the prerequisites for reaching sensitive sinks.

Challenge 1: Difficulty in Identifying Core Parameters for Prioritized Mutation. The core parameter is the user-controllable input that drives the interface operation. Leveraging semantic char-

acteristics to infer the core parameters of operations is the foundation of our methodology. However, IoT interface and parameter names are semantically meaningful but lack consistent naming rules, and a single interface may contain multiple operations. Moreover, it is difficult to efficiently identify the core parameters from a large set of inputs. As a result, traditional NLP methods struggle to identify operations and core parameters from an interface.

Challenge 2: Obstacles in Inferring Operation Prerequisites. In back-end operations, sensitive sinks often lie in specific execution paths and require additional semantic or syntactic validation before payloads can reach them (e.g., an if-condition constrains the user input parameter between 1 and 255). In black-box scenarios, external testing and blind mutations often fail to meet the required prerequisites. This prevents them from reaching target sinks and severely limits coverage depth.

To address these challenges, we propose DetFuzz, an automated fuzzing framework for black-box IoT devices. Initially, we define the operation logic as the atomic semantic label that describes a single, specific operation within an interface. To address Challenge 1, DetFuzz proposes an LLM-based semantic mapping technique to map unstructured interface information to standardized operation logic labels. Based on the operation logic label, DetFuzz further generates the inferred core parameter of this operation. The relevance between the inferred core parameter and each packet parameter is measured by a similarity function. Parameters with high similarity scores form a prioritized mutation candidate pool, which guides the mutation strategy of target parameters. To address Challenge 2, DetFuzz incorporates operation prerequisites into test case construction. It designs a prerequisite reasoning technique with additional front-end analysis information to infer prerequisites. Ultimately, DetFuzz assembles target parameters and these prerequisites into valid test cases.

To evaluate DetFuzz, we conducted systematic experiments on 19 real-world devices from 6 mainstream vendors. The results show that, compared with SOTA black-box and grey-box fuzzing tools, DetFuzz discovered 137 vulnerabilities, an average of 7 times more than these tools, and 122 of them were assigned CVE IDs [35]. DetFuzz achieved 94.48% recall.

In summary, we make the following contributions:

• We present DetFuzz, the first semantic-guided fuzzing framework for IoT devices based on operation logic inference, enabling effective black-box testing on real-world devices. By leveraging observable semantic characteristics, it addresses the lack of information that guides black-box fuzzing.

• We introduce an effective method for generating operation prerequisites by integrating front-end static analysis with LLM-based semantic capabilities. This enables DetFuzz to generate test combinations that satisfy the operation prerequisites.

• Detection of a significant collection of high-risk vulnerabilities. DetFuzz discovered 137 vulnerabilities, including 40 newly assigned as CVE IDs (with an average CVSS 3.x [39] score of 8.54), including buffer overflow and command injection vulnerabilities.

2 Motivation

2.1 Threat Model

DetFuzz operates under a strict real-world black-box threat model. We assume that security analysts cannot obtain the target device's firmware or source code, nor can they perform internal debugging or physically access the target device. In this strict black-box scenario, the analysis relies solely on network reachability to interact with the device's web management interfaces and to open network services. Information about the target device is limited to retrieving front-end elements via web crawling, intercepting data packets with proxy tools, and monitoring device responses. Within this framework, DetFuzz is designed to facilitate the efficient discovery of critical vulnerabilities, specifically buffer overflows and command injections, and increase both the speed and vulnerability discoveries in black-box scenarios for IoT devices.

2.2 Motivating Vulnerabilities

The empirical study reveals that developers tend to assign semantically meaningful names to user-controllable inputs that drive interface operations. These inputs are more likely to flow into sensitive sinks in back-end programs, such as memory operations or command executions, and therefore present higher security risks. A representative example is the `SetPPTPServerCfg` interface in Tenda routers, whose operation is configuring the server IP address. As shown in Figure 1, the core parameters that drive this operation are named `startIp` and `endIp`, while the auxiliary parameter that controls whether the function is enabled is `serverEn`. However, due to missing input length boundary checks, an attacker can trigger a buffer overflow by supplying an excessively long payload in `startIp`. Notably, exploiting this vulnerability requires identifying `startIp` from a large set of inputs and injecting the payload into it, while setting `endIp` to a valid network range and `serverEn` to 1 to enable this operation. Without understanding this relationship, unguided fuzzing has a very low probability of success. Therefore, it is necessary to identify the operation logic and core parameters while analyzing the prerequisites to achieve effective and efficient vulnerability detection.

2.3 Limitations of Existing Techniques

Existing black-box fuzzing tools lack an understanding of the relationships between interface operations and parameters, making it difficult to detect such vulnerabilities effectively. First, tools such as boofuzz [29] and Snipuzz [26] are unable to infer operations from unstandardized interface information or to identify the core parameters. In their mutation strategies, core parameters and auxiliary parameters are treated equally as independent variables. This leads to inefficient allocation of testing resources and decreases the efficiency of vulnerability discovery. Second, existing tools often fail to meet the required prerequisites and therefore cannot generate effective test combinations. Although SRFuzzer [56] can extract front-end restrictions and constraints, it also fails to understand that some parameters require specific prerequisites to trigger vulnerabilities. What is more challenging is that some implicit prerequisites for operations are not indicated in the front-end. In the above example, triggering the vulnerability requires setting `serverEn` to 1 (an operation prerequisite only implied by the parameter name and the front-end code `class= "btn-on"`), assigning `endIp` to a valid network segment (whose validity is verified according to back-end operation rules), and finally injecting the payload into `startIp`. If a fuzzing tool cannot assign specific values to parameters and form

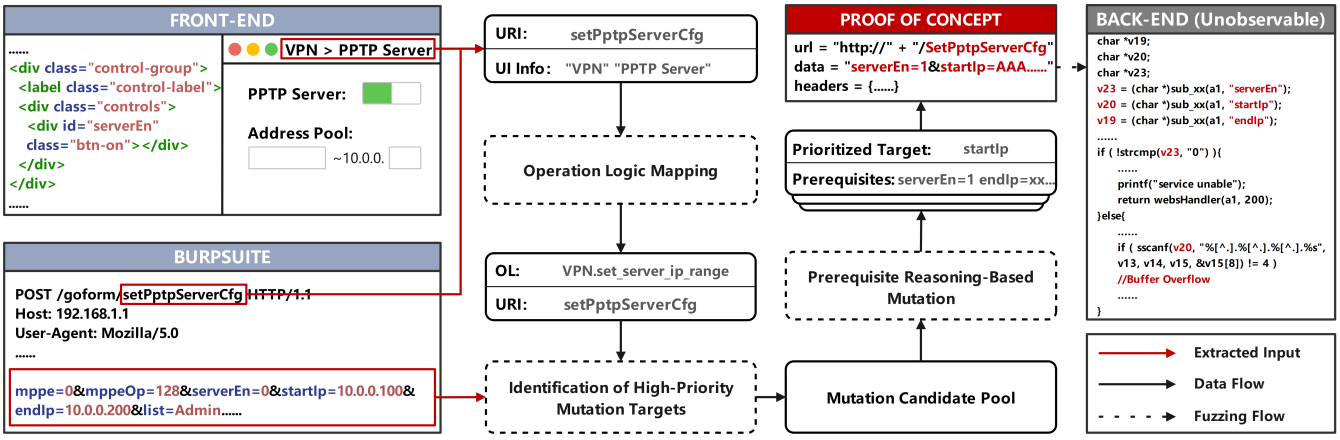


Figure 1: A real-world vulnerability case from an IoT device, serving as a representative example to illustrate DetFuzz's workflow. Similar vulnerabilities affect at least 14 Tenda products. OL: Operation Logic, an atomic semantic label that describes a single, specific operation within an interface.

such combinations, even exhaustive mutation of the core parameter will fail to satisfy the execution prerequisites, resulting in false negatives. Conversely, adopting an all-parameter mutation strategy would cause an exponential explosion of the test space due to the massive number of combinations, making it nearly impossible to hit payloads that satisfy all prerequisites within a limited time.

2.4 DetFuzz

In the `SetPPTPServerCfg` case, using our observations to discover this vulnerability presents two challenges. Challenge 1: How to identify `startIp` as the core parameters among numerous user controllable inputs. Challenge 2: How to infer the prerequisites of `serverEn` and `endIp` needed to implement the operation.

Overview. To address these two challenges, we proposed DetFuzz. DetFuzz uses the observable interface URI, packet parameters, and front-end UI information as input; it outputs valid test cases that both inject payloads into core parameters and satisfy prerequisites. DetFuzz contains three modules. The *Operation Logic Mapping* and *Identification of High-Priority Mutation Targets* modules address Challenge 1, and the *Prerequisite Reasoning-Based Mutation* module addresses Challenge 2.

Given unstructured interface URIs and front-end UI information, the *Operation Logic Mapping* module designs an LLM-based semantic mapping technique to map this information to standardized operation logic labels. Based on the operation logic, the *Identification of High-Priority Mutation Targets* module further uses the LLM to generate the inferred core parameter. The relevance between the inferred core parameter and each packet parameter is measured by a similarity function. Packet parameters with high similarity scores are assigned to a prioritized mutation candidate pool. Guided by this candidate pool, the *Prerequisite Reasoning-Based Mutation* module determines the mutation sequence of target parameters and injects the payloads. It further designs the prerequisite reasoning technique with additional front-end analysis information to infer prerequisites. Ultimately, this module assembles these prerequisites and target parameters into test cases.

Working Example. As shown in Figure 1, we illustrate how DetFuzz addresses the `SetPPTPServerCfg` case in detail.

Operation Logic Mapping. By extracting the interface URI and front-end UI information, DetFuzz uses the LLM-based semantic mapping to classify the interface into the "VPN" category and then map its actual operation to the standardized operation logic label: "Set PPTP Server IP".

Identification of High-Priority Mutation Targets. Based on the operation logic label "Set PPTP Server IP" and URI `SetPPTPServerCfg`, DetFuzz further uses the LLM to generate the inferred core parameter of this operation logic, such as "ip_range". The relevance between the inferred core parameter and each packet parameter is measured by a similarity function. Packet parameters with high similarity scores such as `startIp` and `endIp` are assigned to a prioritized mutation candidate pool.

Prerequisite Reasoning-Based Mutation. Guided by the mutation candidate pool, DetFuzz determines the mutation sequence of target parameters and injects the payloads. For instance, when `startIp` is selected as the target parameter, all remaining parameters including `endIp`, serve as auxiliary parameters; Correspondingly, when `endIp` is selected as the target, `startIp` and the others take on the auxiliary role. Unlike the target, auxiliary parameters are assigned values that aim to satisfy prerequisites. DetFuzz uses prerequisite reasoning with additional front-end analysis information to infer prerequisites. For example, by parsing the HTML `<div id="serverEn" class="btn-on">`, DetFuzz identifies the prerequisite for executing the server IP configuration logic: the `serverEn` parameter must be activated to trigger subsequent operations. Based on this reasoning, DetFuzz infers key-value pairs that satisfy the prerequisite (e.g., `serverEn=1`). Ultimately, DetFuzz assembles target parameters with this prerequisite into valid test cases.

3 Methodology

Based on these ideas, DetFuzz operates in three stages, as shown in Figure 2. It first maps unstructured interface information to a standardized operation logic label. It then identifies high-priority

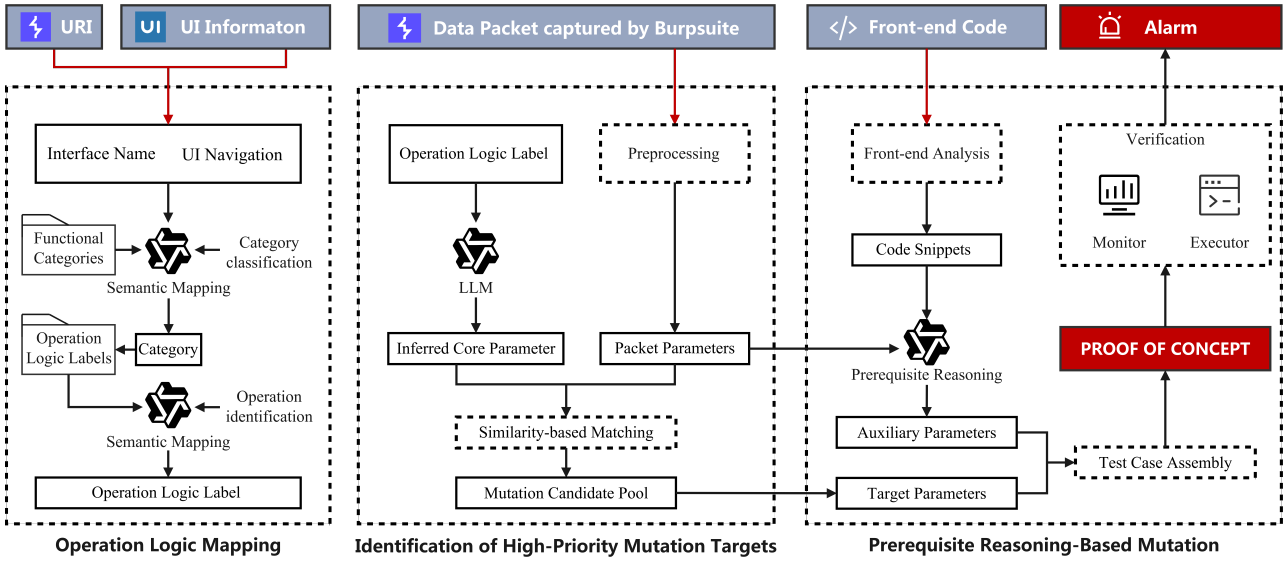


Figure 2: Workflow of DetFuzz. Steps that require the semantic capabilities of the LLM are represented by the Qwen icon.

mutation targets according to operation logic label. Finally, it infers prerequisites and assembles them with the target parameters into valid test cases. We describe each stage below.

3.1 Operation Logic Mapping

This module aims to map unstructured interface information to standardized operation logic. As identified in Challenge 1, IoT interface and parameter names are semantically meaningful but lack consistent naming rules, and a single interface may contain multiple operations. To address this ambiguity, DetFuzz designs an LLM-based semantic mapping technique to identify the interface's operation. Specifically, we established a database of functional categories and operation logic labels to provide external prior knowledge.

Functional Categories. Functional categories partition interfaces according to device management responsibility domains, where each category corresponds to an independent functional subsystem. To construct a representative taxonomy, we selected management interfaces from 46 devices across 9 mainstream IoT vendors, including routers, extenders, IP cameras, and more (specific devices are listed in Appendix B.1). Because devices in the same series exhibit highly consistent management interfaces, we selected one model to represent each series (e.g., TL-WR1043N represents the TL-WR series). First, we collected UI items and functional modules from these management interfaces and organized operations into standard verb-object labels. Second, we used cosine similarity between sentence embeddings to deduplicate redundant labels. Finally, we grouped labels by the UI navigation subpage and the functional modules they belong to. Similar labels are grouped into the same functional category, which requires a one-time manual effort. Appendix B.2 lists the categories (e.g., WiFi, VPN, LAN) and construction details.

Operation Logic. The operation logic label is the atomic semantic label describing a single, specific operation within an interface. At the first step of constructing functional categories, we convert UI items and functional modules into a standardized verb-object structure. As a result, each item forms an independent operation logic label (e.g., `set_usb_ftp_shared_path` in usb category), and these labels constitute the initial operation logic labels database.

This design avoids ambiguity caused by multiple operations within a single interface. For example, an interface named `savePare ntControlInfo` sets access time limits for specific user devices. If the entire interface is modeled as a single operation logic label, both the `device_address` and the `time_limit` may be generated as inferred core parameters. Since these inferred core parameters are semantically heterogeneous, the similarity scores between them and the packet parameters become unreliable. This is because a parameter that is semantically close to `device_address` is unlikely to resemble `time_limit`. More importantly, such mixed targets often break execution paths during testing because they act as prerequisites for one another. Instead, we create two distinct operation logic labels, both in the "Parental Control" category: `parental_control.set_register_device` and `parental_contr ol.set_internet_time_limit`.

Semantic Mapping. The semantic mapping prompt works as follows: By populating the interface URI and front-end UI information, it first guides the LLM to select an appropriate functional category for the interface. Then, operation logic labels belonging to that category are added to the prompt. At the next stage, DetFuzz uses these operation logic labels as external prior knowledge and lets the LLM choose the operation logic labels for the current interface. The prompt design is listed in Appendix B.3.

3.2 Identify High-Priority Mutation Targets

This module aims to identify core parameters that drive the interface's operation logic. As identified in Challenge 1, it is difficult to efficiently identify the core parameters from a large set of inputs. Directly prompting an LLM to select the core parameters from all packet parameters suffers from diluted attention and hal-

lucinations, which hurt robustness. Therefore, DetFuzz designs a similarity-based matching algorithm to identify core parameters.

Similarity-based Matching. As shown in Algorithm 1, it takes the operation logic label, URI, and datapacket parameters as input and outputs the prioritized mutation candidate pool. The datapacket parameters are extracted by the preprocessing step. First, InferCoreParam prompts the LLM to generate the inferred core parameter c based on operation logic label and URI (Line 2). Then, Embed maps c into a vector v_c using an embedding model [6] optimized for code-text semantic alignment (Line 3). For each packet parameter p_i, DetFuzz maps it into a unified vector space as v_i (Line 6) and computes its similarity score s_i via CosineSimilarity (Line 7): $s_i = \text{Sim}(v_c, v_i) = \frac{v_c \cdot v_i}{\|v_c\| \|v_i\|}$. And each pair (p_i, s_i) is added to P (Line 8). Finally, RankByScore ranks P by similarity score (Line 10), and P serves as the prioritized mutation candidate pool.

Algorithm 1 The Cosine Similarity-Based Matching Algorithm

Input: *Operation Logic Label, Datapacket Parameters, URI*
Output: *Prioritized Mutation Candidate Pool P*

1: **procedure** IdentifyCoreParams(Operation Logic Label, URI, Datapacket Parameters)
2: $c \leftarrow$ InferCoreParam(Operation Logic Label, URI)
3: $v_c \leftarrow$ Embed(c)
4: $P \leftarrow \emptyset$
5: **for all** p_i in Datapacket Parameters **do**
6: $v_i \leftarrow$ Embed(p_i)
7: $s_i \leftarrow$ CosineSimilarity(v_c, v_i)
8: $P \leftarrow P \cup \{(p_i, s_i)\}$
9: **end for**
10: $P \leftarrow$ RankByScore(P)
11: **return** P
12: **end procedure**

3.3 Prerequisite Reasoning-Based Mutation

This module aims to generate valid test cases that both inject payloads into the target parameters and satisfy the operation prerequisites, addressing Challenge 2. When prerequisites are required for a specific operation, the search space of testing is significantly massive. This prevents tests from reaching specific execution paths, severely limiting coverage depth. Accordingly, this module designs the prerequisite reasoning technique with additional front-end analysis information to infer prerequisites as auxiliary parameters. It then generates test cases by taking the Cartesian product of the candidate key-value pairs for the target and auxiliary parameters.

Front-end Analysis. It extracts a code window from the target interface's front-end code by matching patterns. Using this code window, it locates identifiers that match packet parameter names, which serve as anchors to capture surrounding code snippets. These code snippets serve as auxiliary information in the few-shot prompt.

Prerequisites Reasoning. It uses the few-shot prompt to guide the LLM to generate reasonable auxiliary parameter values. First, DetFuzz populates the prompt with the code snippets and packet parameters of the target interface. The prompt design assumes that the back-end has strict validation on each parameter and requires the LLM to assign reasonable and valid values to each parameter as much as possible. The LLM needs to identify whether the code snippets contain parameters with binary or multi-valued options (e.g., 0/1, on/off), and if so, rank the values based on the likelihood of successfully executing the operation. For the ordinary auxiliary parameters, the LLM assigns each one reasonable and valid values according to the rules in the prompt.

Test Case Assembly. Specifically, DetFuzz first determines the mutation sequence of target parameters according to the prioritized mutation candidate pool. For target parameter values, DetFuzz injects payload templates while preserving the original value structure. Each target parameter will be tested with both overflow and command-injection templates, and the payload mutation rules include inserting predefined strings and executing file-writing commands (details in section 4.1.4). When a parameter is selected as the target parameter, only it is injected with payloads, while all remaining parameters serve as auxiliary parameters and are filled with the values generated in the prerequisite reasoning stage. If any auxiliary parameter has multi-valued options, its values are tested sequentially in the given ranking order. Ultimately, DetFuzz assembles target parameters with auxiliary parameters into test cases. To maintain protocol consistency, we decouple protocol header fields from the packet parameters. Whenever the parameters are modified, the module recalculates and updates header fields, ensuring that the server can correctly parse malformed payloads.

4 Evaluation

4.1 Experiment Setup

In this section, we provide a comprehensive overview of the experimental setup to evaluate DetFuzz.

4.1.1 Experiment Environment and Device Selection. All experiments were conducted on a Windows 11 host machine equipped with an Intel Core i7-1260P CPU (12 cores, 16 threads) and 40GB of RAM. The core DetFuzz modules, including interface analysis and payload generation, run directly on the host system. Vulnerability verification and monitoring were performed on an Ubuntu 22.04 LTS virtual machine hosted in VMware Workstation Pro 17, with 4 CPU cores and 16GB of RAM. All tools used in our comparison experiments were run in the same virtual machine. Following established fuzzing methodologies, we set a 24-hour execution timeout for all evaluated tools. To ensure the authenticity and stability of the results, we verified any payload that successfully triggered a vulnerability through 10 rounds of verification.

4.1.2 DetFuzz Evaluation Dataset. As shown in Table 1, we selected 19 devices from six mainstream IoT vendors for vulnerability detection, aligning with top-tier work's evaluation [22, 23, 26, 41, 57, 58] and market reports [4, 8, 9, 17]. The device types include routers, extenders, IP cameras, and media bridges [10, 15, 16]. The evaluation targeted buffer overflow and command injection vulnerabilities, as these types are the mainstream and highest-risk flaws [3, 12].

For devices with no risk of damage, as well as devices whose firmware is encrypted and cannot be extracted, such as Tenda AC21 and AC23, we verified directly on physical machines. To ensure controllability and reproducibility, vulnerability testing that posed high risk of device damage was conducted on remote QEMU [18] emulation environments that replicate the physical target devices.

Table 1: DetFuzz Vulnerability Detection Experiment Result

Device ID	Vendor	Product	Type	Interfaces	Test Units	Vuls	RC	TP	Time(s)	Env
1	Tenda	AC21	Router	37	305	12	12	12	1954.15	Physical Machine
2	Tenda	AC23	Router	37	283	15	15	15	2496.88	Physical Machine
3	Tenda	AC15	Router	38	266	19	19	19	2314.28	Remote
4	Tenda	AC18	Router	38	280	19	19	19	2021.77	Remote
5	Tenda	AC10U	Router	37	261	16	16	16	2073.13	Remote
6	Linksys	RE6500	Extender	8	19	1	1	1	425.91	Physical Machine
7	Linksys	RE7000	Extender	8	22	1	1	1	410.24	Physical Machine
8	Linksys	RE9000	Extender	8	19	1	1	1	411.30	Physical Machine
9	D-Link	DCS-930L	IP Camera	11	122	1	1	1	1283.09	Physical Machine
10	D-Link	DCS-932L	IP Camera	12	153	1	1	1	1373.62	Physical Machine
11	Trendnet	TEW800	Media Bridge	5	28	1	1	1	306.81	Physical Machine
12	TP-Link	wr740nv6	Router	34	365	2	2	2	2552.45	Physical Machine
13	TP-Link	wr842ndv2	Router	38	332	2	2	2	2209.92	Physical Machine
14	TP-Link	wr841ndv11	Router	34	388	2	2	2	2361.16	Physical Machine
15	TP-Link	wr941ndv6	Router	34	372	1	1	1	2129.09	Physical Machine
16	TP-Link	wr949nv6	Router	35	359	1	1	1	2006.45	Physical Machine
17	TOTOLINK	LR350	Router	28	237	13	13	13	1722.64	Remote
18	TOTOLINK	NR1800X	Router	29	235	14	14	14	1890.00	Remote
19	TOTOLINK	A3700R	Router	33	572	15	15	15	2391.66	Remote
Total	–	–	–	**504**	**4618**	**137**	**137**	**137**	**8h59min**	–

Interfaces: Number of tested device interfaces; **Env:** The experimental testing environment.
Vuls: Vulnerabilities discovered by DetFuzz; **TP:** Manually verified true positive vulnerabilities.

For instance, we observed that the buffer overflow vulnerability CVE-2024-42545 [13] caused a fatal crash on a TOTOLINK A3700R, rendering the device unrecoverable even after a physical reset. It must be emphasized that, under all testing conditions, DetFuzz relies exclusively on information observable in a strict black-box scenario, which aligns with our defined threat model.

4.1.3 Implementation of DetFuzz. DetFuzz is implemented in Pytho n 3.11 and includes the code implementing our methodology, along with an initial JSON database. We use qwen3-max [52] as the core LLM, with temperature set to 0.

For DetFuzz's core input, upon logging into the IoT device's management interface, we use Selenium 4.31.0 [5] to crawl the front-end source code and apply regular expressions matching to identify critical code segments, such as HTML forms. We use Burp Suite Professional v2023.4.5 [2] as an HTTP/HTTPS proxy to capture request and response packets triggered by interactions, including button clicks and form submissions. For the core components, we follow the criteria in Section 3.1 to build the functional categories and the operation logic database. We use the OpenAI text-embedding-3-large [6] model to compute the cosine similarity between the packet parameters and the inferred core parameters. As a limitation, DetFuzz requires only two manual steps: initial page login and UI extraction setup. For each new device, login and packet capture typically take 30 seconds, and UI extraction takes 5 minutes.

4.1.4 Automated Payload Execution and Monitoring Module. The automated payload execution and monitoring module achieves high accuracy through a well-designed monitoring mechanism, which can be categorized into two groups based on vulnerability type.

For buffer overflow, the monitoring module detects crashes of critical services (e.g., the HTTP daemon), which usually make the affected device's IP address inaccessible. To ensure the crash is triggered, we first replay the original request and then replace only the identified target parameter with an overlong string (2000*A). A deterministic crash under this isolated mutation is treated as overflow rather than null-dereference or environmental issues.

For command injection, the execution module first attempts to write a marker file (e.g., hacker.txt) to the device's web root directory, and then the monitoring module verifies its presence via an HTTP GET request. A 200 OK response confirms successful execution, while a 404 NOT FOUND indicates failure. If successful, the execution module issues a follow-up payload to remove the marker file, restoring the environment to a clean state for subsequent tests.

4.2 Vulnerability Detection Experiment

Table 1 presents results from automated vulnerability discovery experiments on 19 target devices. The term "Time" represents the cumulative duration required for LLM reasoning, the similarity function, and final payload generation. For the 504 total interfaces, DetFuzz determines the mutation sequence of target parameters, injects the payloads, and then generates the operation prerequisites. The payload generation strategy is defined as the Cartesian product between the candidate key-values of the target parameters and auxiliary parameters, which are then populated into predefined PoC templates. For target parameter values, DetFuzz injects {overflow}/{cmdi} templates according to the testing queue. During execution, DetFuzz populates payloads into vendor-specific Python script templates that use the requests module to send POST requests to the target device at its designated IP address.

Table 2: Comparison Results of Vulnerability Discovery and Recall

Vendor	Devices	DetFuzz	boofuzz	Snipuzz	FirmAFL	Greenhouse	Total
Tenda	5	**81 (100.00%)**	15 (18.52%)	10 (12.35%)	\	\	81
TP-Link	5	8 (66.67%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	8 (66.67%)	12
TOTOLINK	3	**42 (100.00%)**	2 (4.76%)	0 (0.00%)	\	\	42
Linksys	3	**3 (100.00%)**	1 (33.33%)	1 (33.33%)	\	\	3
D-Link	2	2 (33.33%)	0 (0.00%)	0 (0.00%)	\	**4 (66.67%)**	6
TrendNet	1	**1 (100.00%)**	0 (0.00%)	0 (0.00%)	\	0 (0.00%)	1
SUM	**19**	**137 (94.48%)**	**18 (12.41%)**	**11 (7.59%)**	**0 (0.00%)**	**12 (8.28%)**	**145**

Note: Fuzzing tool columns represent "Vulnerabilities (Recall%)". "\" represents the grey-box testing tools that do not support firmware emulation, rendering testing infeasible. **Total** represents the sum of root cause vulnerabilities discovered by all evaluated tools.

We define a unique triple ⟨Interface, Target Parameter, Prerequisites⟩ as one test unit, and a unique binary tuple ⟨Interface, Target Parameter⟩ as the root cause vulnerability (RC). If the monitoring module reports that a specific test unit triggered a vulnerability alert, the binary tuple it belongs to is confirmed as a root cause vulnerability. To eliminate redundancy, the system automatically terminates testing of other triples that share the same binary tuple but have different prerequisites. We set a maximum duration of 1,000 seconds per test unit, ensuring that the total testing time remains within the 24-hour limit. By leveraging this payload testing logic alongside the monitoring module, DetFuzz efficiently discovered 137 vulnerabilities. These include 125 overflow vulnerabilities and 12 injection vulnerabilities. Among these vulnerabilities, 55 were previously unknown; 40 of them have been assigned CVE IDs. The other 12 vulnerabilities are currently under review, and the remaining 3 vulnerabilities are not assigned CVE IDs because their target products have reached End-of-Life. The CVEs we found are listed in Appendix C in Table 5. We also provide an end-to-end execution trace in Appendix C.1 as a recommended workflow for reviewers to apply DetFuzz to a new device.

4.3 Comparison Experiment

We selected four open-source SOTA tools for comparison. Among them, boofuzz [29] and snipuzz [26] share DetFuzz's threat model. Given that contemporary grey-box fuzzing tools still face significant challenges with low-fidelity firmware emulation, we also included FirmAFL [59] and Greenhouse [47] to provide a more comprehensive assessment of DetFuzz's efficacy in real-world IoT testing scenarios.

The comparison experiments primarily focused on two metrics: the number of discovered vulnerabilities and the recall rate. Evaluating recall in a strict black-box environment is challenging because the absolute ground truth (i.e., the total number of existing vulnerabilities in a device) cannot be pre-determined. Therefore, we adopted a pragmatic methodology: the union of unique vulnerabilities discovered by all five evaluated tools was defined as the ground truth for this experiment, aligning with FirmRec [51] and IoTBec [53]. Accordingly, we measured each tool's recall as the proportion of vulnerabilities it successfully identified within this aggregate set. Table 2 provides a detailed comparison of the vulnerability discovery capabilities and recall rates between DetFuzz and existing black-box and grey-box fuzzing tools.

The results show that DetFuzz discovers more than 7 times as many vulnerabilities as other SOTA tools and achieves 94.48% recall. Additionally, DetFuzz is much more efficient than the baseline. DetFuzz's total run time is 8h59min, discovering 15.25 vulnerabilities per hour; the other four tools run for the full 24 hours: boofuzz discovers 0.75 per hour, snipuzz discovers 0.46 per hour, and greenhouse discovers 0.50 per hour. A single complete experimental run on 19 devices of DetFuzz costs only 12.44 USD, and the average cost to find a vulnerability is 0.09 USD.

4.4 Ablation Experiment

To evaluate the independent contributions of DetFuzz's components, we design ablation experiments targeting its core capabilities. DetFuzz consists of three main modules. Operation Logic Mapping provides the semantic foundation for Identification of High-Priority Mutation Targets. Therefore, DetFuzz has two essential capabilities:

- Identifying core parameters based on observable semantic information to guide fuzzing strategy.
- Infer prerequisites required to implement the operations.

The ablation study uses the full DetFuzz system as the baseline. Ablation-1 removes the capability of identifying core parameters and instead selects mutation targets randomly from the parameter space. This setting evaluates the impact of Operation Logic Mapping and Identification of High-Priority Mutation Targets. Ablation-2 removes the capability of inferring prerequisites during the mutation stage. Attack payloads are directly injected into target parameters without satisfying prerequisites.

Experimental results in Table 3 show that Ablation-1 detects only 51.09% of the vulnerabilities compared to the baseline system, and Ablation-2 detects only 71.53% of the vulnerabilities detected by the baseline system. This significant degradation shows that both capabilities are essential to improving the efficiency of vulnerability discovery. Without identifying core parameters, testing resources are widely dispersed across the parameter space. Without analyzing prerequisites, operation execution paths become unreachable, preventing potential vulnerabilities from being effectively triggered.

5 Related Work

5.1 Fuzzing-based Dynamic Analysis

Dynamic analysis, represented by fuzzing, is a mainstream method for IoT security to detect program behavior during execution.

Table 3: Ablation Experiment on DetFuzz framework

Vendor	Vulnerabilities			Time		
	DetFuzz	Ablation-1	Ablation-2	Time	Time-1	Time-2
Tenda	81	48	61	3h1min	1h47min	2h17min
TP-Link	8	0	4	3h7min	1h51min	2h23min
TOTOLINK	42	18	27	1h41min	1h1min	1h17min
Linksys	3	3	3	21min	13min	16min
D-Link	2	1	2	44min	27min	33min
TrendNet	1	0	1	5min	3min	4min
SUM	**137**	**70 (51.09%)**	**98 (71.53%)**	**8h59min**	**5h22min**	**6h50min**

Ablation-1 removes the ability to identify core parameters and randomly selects from inputs; Ablation-2 removes prerequisite analysis, injecting directly into the target.

5.1.1 Black-box Fuzzing. Fuzzing is the primary method for detecting vulnerabilities in black-box IoT devices. Boofuzz [29] provides automated frameworks for network protocol fuzzing and often serves as a benchmark. Snipuzz [26] uses response differences to infer the snippet message format and uses this information to optimize mutation without firmware. However, IoT device responses are usually simple and may not provide enough information. SRFuzzer [56] and ESRFuzzer [55] specifically target SOHO routers by generating seed data in key-value format and mutating it according to rule-based templates. Nevertheless, the reliance on predefined mutation rules still limits their ability to discover vulnerabilities. IoTFuzzer [21] uses mobile apps for dynamic analysis and mutates the communication traffic. However, app-side sanitizers may prevent mutated payloads from reaching the target. DIANE [42] attempts to bypass the sanitizers by using code snippets captured after validation but before transformation. It is useful but may destroy original data formats and reduce mutation efficiency. HubFuzzer [33] mutates only critical fields by learning the protocol grammar, ensuring message-format legality. LABRADOR [30] leverages string information from device responses to infer internal firmware execution paths, helping discover vulnerabilities. Despite its efficiency, it still falls short of black-box testing because it relies on firmware analysis.

5.1.2 Grey-box Fuzzing. For grey-box fuzzing, FirmAFL targets embedded firmware and can efficiently discover vulnerabilities through QEMU-based device emulation. EM-Fuzz [28] and IoTHunter [54] both implanted real-time checkers in the emulation environment to assist in fuzzing. TriforceAFL [37] provides coverage-guided kernel fuzzing via a full-system emulator, and subsequent efforts accommodate IoT devices based on this approach. Greenhouse [47] provides a high-fidelity rehosting environment that decouples firmware execution from specific hardware dependencies. HouseFuzz [50] introduces a service-aware approach that identifies and uses complex dependencies among firmware services. These methods are useful but require firmware emulation, and full-system emulation and external interactions are complex [20, 49]. Additionally, their high execution overhead makes them difficult to deploy on resource-constrained IoT devices [1, 7, 46].

5.2 Static Analysis

Static taint analysis is a mainstream approach for identifying potential security vulnerabilities without executing the program. Typical static analysis methods include symbolic execution and taint analysis. KARONTE [43] performs static taint tracking by analyzing interactions between multiple binaries to identify vulnerabilities. SaTC [22] executes taint analysis by matching shared parameter keywords between front-end web files and back-end binaries. FITS [31] infers taint sources to improve analysis accuracy, and Bridge [41] constructs inter-process relationships to detect high-order taint vulnerabilities. LARA [57] introduces LLMs to infer semantic relationships between code and data, broadening the scope of taint sources by identifying hidden interfaces. NÜWA [58] uses semantic constraints to detect vulnerabilities. By leveraging LLM semantic reasoning, it analyzes and verifies semantic differences in constraint logic to pinpoint exploitable "beacons". In summary, while these taint analysis methods use LLMs to analyze semantic information and achieve advances, they all depend on acquiring and unpacking firmware rather than using semantic information in a black box.

6 Discussion

This section discusses the dependencies and limitations of DetFuzz.

1. Detection Scope. DetFuzz primarily targets HTTP services on devices accessible to testers and uses captured requests as starting inputs because HTTP is the mainstream protocol for IoT device management [14]. It can also handle devices that use other protocols, such as MQTT. We conducted an extra experiment on the TOTOLINK T6 in our artifact and found 3 command injection vulnerabilities to demonstrate its ability in MQTT-driven devices.

UI items and front-end code can provide additional semantic information and prerequisites, but they are not necessary inputs. For example, the analysis of the three TOTOLINK devices used only interface and parameter names from captured requests as required input, without front-end code, and still uncovered 42 vulnerabilities.

In conclusion, reachable semantic requests are fundamental to DetFuzz. Devices that do not support semantic requests are outside the current scope, which are the primary focus of these works [21, 33, 42]. The practicality of DetFuzz is not affected by the client type or protocol used by the device (e.g., browser or mobile app).

2. Dependency of LLMs. DetFuzz relies on the LLM's semantic capability. In addition to Qwen3, we also tested DeepSeek V3.2 [25] and OpenAI GPT-4-0613 [40], both of which were comparably effective. GPT-4-0613 detected 136 vulnerabilities, and DeepSeek V3.2 detected 134, showing the method's robustness using different LLMs. For resource billing, a single complete experimental run on 19 devices costs only 12.44 USD, and the average cost to find a vulnerability is 0.09 USD. Taking Table 1 Tenda AC10U as an example, the 37 accessible interfaces of the test device consumed a total of 317,576 tokens, averaging 8,583 tokens per interface.

7 Conclusion

This paper proposes DetFuzz, a novel fuzzing framework for IoT vulnerability detection with no reliance on firmware or source code. It uses semantic characteristics present in observable information under black-box scenarios to infer the actual relationships between interface operations and parameters in back-end code. By prioritizing fuzzing resources to user-controllable inputs that drive operations, DetFuzz significantly improved vulnerability discovery efficiency. Experiments demonstrate that DetFuzz outperforms current black-box and grey-box SOTA methods in discovering vulnerabilities. It has also discovered 40 new high-risk vulnerabilities, which have been assigned CVE IDs.

References

[1] [n. d.]. AddressSanitizer (ASan) — C/C++ Sanitizer Support in MSVC. [Online]. Available: https://learn.microsoft.com/zh-cn/cpp/sanitizers/asan?view=msvc-170.

[2] [n. d.]. Burp Suite Professional / Community Release 2023.4.5. [Online]. Available: https://portswigger.net/burp/releases/professional-community-2023-4-5.

[3] [n. d.]. Every 24 Hours, Home Networks See an Average of 10 Attacks. Is Your Network Secure? [Online]. Available: https://www.netgear.com/hub/network/2024-iot-threat-report/.

[4] [n. d.]. Global Wireless Router Market Size And Forecast. [Online]. Available: https://www.verifiedmarketresearch.com/product/wireless-router-market/.

[5] [n. d.]. Selenium — Python Package Index. [Online]. Available: https://pypi.org/project/selenium/.

[6] [n. d.]. text-embedding-3-large Model Documentation. [Online]. Available: https://platform.openai.com/docs/models/text-embedding-3-large.

[7] [n. d.]. UndefinedBehaviorSanitizer (UBSan) — Clang Documentation. [Online]. Available: https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html.

[8] [n. d.]. Wireless Router Market Size, Share and Industry Analysis. [Online]. Available: https://www.fortunebusinessinsights.com/wireless-router-market-107203.

[9] [n. d.]. Wireless Routers Market - Global Industry Research Analysis. [Online]. Available: https://www.marketresearchstore.com/market-insights/wireless-routers-market-784987.

[10] [n. d.]. Zscaler ThreatLabz 2025 Mobile, IoT & OT Threat Report. [Online]. Available: https://www.zscaler.com/campaign/threatlabz-mobile-iot-ot-report.

[11] 2020. ETSI EN 303 645 V2.1.1: Cyber Security for Consumer Internet of Things – Baseline Requirements. [Online]. Available: https://www.etsi.org/deliver/etsi_en/303600_303699/303645/02.01.01_60/en_303645v020101p.pdf.

[12] 2024. 2023 Top Routinely Exploited Vulnerabilities – Joint Cybersecurity Advisory. [Online]. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa24-317a.

[13] 2024. CVE-2024-42545: Buffer Overflow in TOTOLINK A3700R Router. [Online]. Available: https://www.cve.org/CVERecord?id=CVE-2024-42545.

[14] 2024. IoT Standards and Interoperability: Enabling Seamless Communication and Integration in Heterogeneous Environments. (2024). [Online]. Available: https://admin.mantechpublications.com/index.php/JoITIT/issue/viewFile/6482/7083.

[15] 2024. IoT Vulnerabilities Skyrocket, Becoming Key Entry Point for Attackers. [Online]. Available: https://www.infosecurity-magazine.com/news/iot-vulnerabilities-entry-point/.

[16] 2025. Routers Remain Top Target for IoT Attacks, Per Zscaler Report. [Online]. Available: https://www.admin-magazine.com/News/Routers-Remain-Top-Target-for-IoT-Attacks-Per-Zscaler-Report.

[17] 2025. U.S. conducting criminal antitrust investigation into TP-Link, Bloomberg News reports. [Online]. Available: https://www.reuters.com/technology/tp-link-faces-us-criminal-antitrust-investigation-bloomberg-news-reports-2025-04-25/.

[18] Fabrice Bellard. 2005. QEMU, a Fast and Portable Dynamic Translator. In *2005 USENIX Annual Technical Conference (USENIX ATC 05)*. Anaheim, CA, 41–46. [Online]. Available: https://www.usenix.org/conference/2005-usenix-annual-technical-conference/qemu-fast-and-portable-dynamic-translator.

[19] Kaitlin Boeckl, Michael Fagan, William Fisher, Naomi Lefkovitz, Katerina N. Megas, Ellen Nadeau, Ben Piccarreta, Danna Gabel O'Rourke, and Karen Scarfone. 2019. *Considerations for Managing Internet of Things (IoT) Cybersecurity and Privacy Risks*. NIST Interagency/Internal Report (NISTIR) 8228. National Institute of Standards and Technology (NIST). [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ir/2019/nist.ir.8228.pdf.

[20] Daming D. Chen, Manuel Egele, Maverick Woo, and David Brumley. 2016. Towards Automated Dynamic Analysis for Linux-based Embedded Firmware. In *Proceedings of the Network and Distributed System Security Symposium (NDSS) 2016*. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2017/09/towards-automated-dynamic-analysis-linux-based-embedded-firmware.pdf.

[21] Jiongyi Chen, Wenrui Diao, Qingchuan Zhao, Chaoshun Zuo, Zhiqiang Lin, XiaoFeng Wang, Wing Cheong Lau, Menghan Sun, Ronghai Yang, and Kehuan Zhang. 2018. IOTFUZZER: Discovering Memory Corruptions in IoT Through App-based Fuzzing. In *Network and Distributed System Security (NDSS) Symposium 2018*. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_01A-1_Chen_paper.pdf.

[22] Libo Chen, Yanhao Wang, Quanpu Cai, Yunfan Zhan, Hong Hu, Jiaqi Linghu, Qinsheng Hou, Chao Zhang, Haixin Duan, and Zhi Xue. 2021. Sharing More and Checking Less: Leveraging Common Input Keywords to Detect Bugs in Embedded Systems. In *30th USENIX Security Symposium (USENIX Security 21)*. 303–319. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/chen-libo.

[23] Kai Cheng, Yaowen Zheng, Tao Liu, Le Guan, Peng Liu, Hong Li, Hongsong Zhu, Kejiang Ye, and Limin Sun. 2023. Detecting Vulnerabilities in Linux-Based Embedded Firmware with SSE-Based On-Demand Alias Analysis. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) *(ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 360–372. doi:10.1145/3597926.3598062

[24] Barbara Cuthill. 2021. Whether You Build Them or Buy Them – IoT Device Security Concerns Us All. [Online]. Available: https://www.nist.gov/blogs/manufacturing-innovation-blog/whether-you-build-them-or-buy-them-iot-device-security-concerns.

[25] DeepSeek AI. 2025. DeepSeek V3.2 Release. https://api-docs.deepseek.com/news/news251201. https://api-docs.deepseek.com/news/news251201 DeepSeek-V3.2: Reasoning-first model; official model release and tech report available.

[26] Xiaotao Feng, Ruoxi Sun, Xiaogang Zhu, Minhui Xue, Sheng Wen, Dongxi Liu, Surya Nepal, and Yang Xiang. 2021. Snipuzz: Black-box Fuzzing of IoT Firmware via Message Snippet Inference. *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security* (2021), 337–350. [Online]. Available: https://arxiv.org/pdf/2105.05445.

[27] Guangming Gao, Shuitao Gan, Xiaofeng Wang, and Shengkai Zhu. 2024. LLMUZZ: LLM-based seed optimization for black-box device fuzzing. In *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 1209–1216. doi:10.1109/trustcom63139.2024.00171 [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10945143/.

[28] Jian Gao, Yiwen Xu, Yu Jiang, Zhe Liu, Wanli Chang, Xun Jiao, and Jiaguang Sun. 2020. EM-Fuzz: Augmented Firmware Fuzzing via Memory Checking. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 3420–3432. doi:10.1109/TCAD.2020.3013046 [Online]. Available: https://ieeexplore.ieee.org/document/9211428/.

[29] jtpereyda. 2025. boofuzz: A fork and successor of the Sulley Fuzzing Framework. [Online]. Available: https://github.com/jtpereyda/boofuzz.

[30] Hangtian Liu, Shuitao Gan, Chao Zhang, Zicong Gao, Hongqi Zhang, Xiangzhi Wang, and Guangming Gao. 2024. Labrador: Response Guided Directed Fuzzing for Black-box IoT Devices. In *2024 IEEE Symposium on Security and Privacy (SP)*. 1920–1938. doi:10.1109/sp54263.2024.00127 [Online]. Available: https://ieeexplore.ieee.org/document/10646723.

[31] Puzhuo Liu, Yaowen Zheng, Chengnian Sun, Chuan Qin, Dongliang Fang, Mingdong Liu, and Limin Sun. 2023. FITS: Inferring Intermediate Taint Sources for Effective Vulnerability Analysis of IoT Device Firmware. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. 138–152.

[32] Xiaoyue Ma, Lannan Luo, and Qiang Zeng. 2024. From One Thousand Pages of Specification to Unveiling Hidden Bugs: Large Language Model Assisted Fuzzing of Matter IoT Devices. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4783–4800. [Online]. Available: https://www.usenix.org/system/files/usenixsecurity24-ma-xiaoyue.pdf.

[33] Xiaoyue Ma, Qiang Zeng, Haotian Chi, and Lannan Luo. 2023. No More Companion Apps Hacking but One Dongle: Hub-Based Blackbox Fuzzing of IoT Firmware. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications, and Services (MobiSys 2023)*. 205–218. doi:10.1145/3581791.3596857 [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3581791.3596857.

[34] Katerina N. Megas, Barbara Cuthill, and Sarbari Gupta. 2021. *Establishing Confidence in IoT Device Security: How do we get there? (Draft)*. NIST Cybersecurity White Paper (Draft) CSWP.05142021-draft. National Institute of Standards and Technology (NIST). [Online]. Available: https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.05142021-draft.pdf.

[35] MITRE. 2025. CVE® – Common Vulnerabilities and Exposures. [Online]. Available: https://cve.mitre.org/.

[36] NCC Group. 2025. Enterprise Connected Device Vulnerability Assessment. [Online]. Available: https://assets.publishing.service.gov.uk/media/681de9fdd9c9bb76078f7e82/Enterprise_Connected_Device_Vulnerability_Assessment_-_NCC_Group.pdf.

[37] Nccgroup. 2016. TriforceAFL: AFL/QEMU fuzzing with full-system emulation. https://github.com/nccgroup/TriforceAFL Online.

[38] Nicolas Nino, Ruibo Lu, Wei Zhou, Kyu Hyung Lee, Ziming Zhao, and Le Guan. 2024. Unveiling IoT Security in Reality: A Firmware-Centric Journey. In *33rd USENIX Security Symposium (USENIX Security 24)*. 5609–5626. [Online]. Available: https://www.usenix.org/system/files/usenixsecurity24-nino.pdf.

[39] NIST. 2025. NVD CVSS v3 Calculator. [Online]. Available: https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator.

[40] OpenAI. 2026. GPT-4-0613 Model. https://platform.openai.com/docs/models/gpt-4. https://platform.openai.com/docs/models/gpt-4 OpenAI GPT-4 snapshot model as listed in the official API documentation.

[41] Jiaqian Peng, Puzhuo Liu, Yicheng Zeng, Kai Cheng, Yongji Liu, Yun Yang, and Hongsong Zhu. 2026. Bridge: High-Order Taint Vulnerabilities Detection in Linux-based IoT Firmware. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3320–3339. https://www.computer.org/csdl/proceedings-article/sp/2026/606500a001/2bojv5U1cE8

[42] Nilo Redini, Andrea Continella, Dipanjan Das, Giulio De Pasquale, Noah Spahn, Aravind Machiry, Antonio Bianchi, Christopher Kruegel, and Giovanni Vigna. 2021. DIANE: Identifying Fuzzing Triggers in Apps to Generate Under-constrained Inputs for IoT Devices. In *Proceedings of the IEEE Symposium on*

Security & Privacy (S&P). 484–500. doi:10.1109/SP40001.2021.00066 [Online]. Available: https://ieeexplore.ieee.org/document/9519432.

[43] Nilo Redini, Aravind Machiry, Ruoyu Wang, Chad Spensky, Andrea Continella, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2020. Karonte: Detecting Insecure Multi-binary Interactions in Embedded Firmware. In *2020 IEEE Symposium on Security and Privacy (SP)*. 1544–1561. doi:10.1109/SP40000.2020.00036 [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9152796/.

[44] S. Sinha. 2025. 6 IoT semiconductor predictions for 2026. [Online]. Available: https://iot-analytics.com/iot-semiconductor-predictions/.

[45] S. Sinha. 2025. State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally. [Online]. Available: https://iot-analytics.com/number-connected-iot-devices/.

[46] Dokyung Song, Julian Lettner, Prabhu Rajasekaran, Yeoul Na, Stijn Volckaert, Per Larsen, and Michael Franz. 2019. SoK: Sanitizing for Security. In *2019 IEEE Symposium on Security and Privacy (SP)*. 1275–1295. doi:10.1109/SP.2019.00010 [Online]. Available: https://ieeexplore.ieee.org/abstract/document/8835294/.

[47] Hui Jun Tay, Kyle Zeng, Jayakrishna Menon Vadayath, Arvind S. Raj, Audrey Dutcher, Tejesh Reddy, Wil Gibbs, Zion Leonahenahe Basque, Fangzhou Dong, Zack Smith, Adam Doupé, Tiffany Bao, Yan Shoshitaishvili, and Ruoyu Wang. 2023. Greenhouse: Single-Service Rehosting of Linux-Based Firmware Binaries in User-Space Emulation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5791–5808. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/tay.

[48] The MITRE Corporation. 2024. CVE-2024-7175: TOTOLINK A3600R router stack overflow. [https://nvd.nist.gov/vuln/detail/cve-2024-7175.

[49] Jayakrishna Vadayath, Moritz Eckert, Kyle Zeng, Nicolaas Weideman, Gokulkrishna Praveen Menon, Yanick Fratantonio, Davide Balzarotti, Adam Doupé, Tiffany Bao, Ruoyu Wang, Christophe Hauser, and Yan Shoshitaishvili. 2022. Arbiter: Bridging the Static and Dynamic Divide in Vulnerability Discovery on Binary Programs. In *31st USENIX Security Symposium (USENIX Security 22)*. 413–430. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/vadayath.

[50] Haoyu Xiao, Ziqi Wei, Jiarun Dai, Bowen Li, Yuan Zhang, and Min Yang. 2025. HouseFuzz: Service-Aware Grey-Box Fuzzing for Vulnerability Detection in Linux-Based Firmware. In *Proceedings of the 2025 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 3801–3819. doi:10.1109/SP61157.2025.00213

[51] Haoyu Xiao, Yuan Zhang, Minghang Shen, Chaoyang Lin, Can Zhang, Shengli Liu, and Min Yang. 2024. Accurate and efficient recurring vulnerability detection for iot firmware. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3317–3331.

[52] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Ruize Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Mingfeng Yu, Lianghao Deng, Mei Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Xinyu Wang, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. [Online]. Available: https://arxiv.org/abs/2505.09388.

[53] Haoran Yang, Jiaming Guo, Shuangning Yang, Guoli Zhao, Qingqi Liu, Chi Zhang, Zhenlu Tan, Lixiao Shan, Qihang Zhou, Mengting Zhou, et al. 2026. IoTBec: An Accurate and Efficient Recurring Vulnerability Detection Framework for Black Box IoT devices.. In *Proceedings of the Network and Distributed System Security Symposium (NDSS) 2026*.

[54] Bo Yu, Pengfei Wang, Tai Yue, and Yong Tang. 2019. Poster: Fuzzing IoT Firmware via Multi-stage Message Generation. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*. ACM, London, United Kingdom, 2525–2527. doi:10.1145/3319535.3363247

[55] Yu Zhang, Wei Huo, Kunpeng Jian, Ji Shi, Longquan Liu, Yanyan Zou, and Baoxu Liu. 2021. ESRFuzzer: an enhanced fuzzing framework for physical SOHO router devices to discover multi-type vulnerabilities. *Cybersecurity* 4 (2021), 24. doi:10.1186/s42400-021-00091-9 [Online]. Available: https://link.springer.com/article/10.1186/s42400-021-00091-9.

[56] Yu Zhang, Wei Huo, Kunpeng Jian, Ji Shi, Haoliang Lu, Longquan Liu, Chen Wang, Dandan Sun, Chao Zhang, and Baoxu Liu. 2019. SRFuzzer: An Automatic Fuzzing Framework for Physical SOHO Router Devices to Discover Multi-Type Vulnerabilities. In *Proceedings of the 35th Annual Computer Security Applications Conference (ACSAC 2019)*. 544–556. doi:10.1145/3359789.3359826 [Online]. Available: https://doi.org/10.1145/3359789.3359826.

[57] Jiaxu Zhao, Yuekang Li, Yanyan Zou, Zhaohui Liang, Yang Xiao, Yeting Li, Bingwei Peng, Nanyu Zhong, Xinyi Wang, Wei Wang, and Wei Huo. 2024. Leveraging Semantic Relations in Code and Data to Enhance Taint Analysis of Embedded Systems. In *33rd USENIX Security Symposium (USENIX Security 24)*. 7067–7084. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/zhao.

[58] Jiaxu Zhao, Yuekang Li, Yanyan Zou, Yang Xiao, Naijia Jiang, Yeting Li, Nanyu Zhong, Bingwei Peng, Kunpeng Jian, and Wei Huo. 2025. From Constraints to Cracks: Constraint Semantic Inconsistencies as Vulnerability Beacons for Embedded Systems. In *34th USENIX Security Symposium (USENIX Security 25)*. 685–704. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/zhao.

[59] Yaowen Zheng, Ali Davanian, Heng Yin, Chengyu Song, Hongsong Zhu, and Limin Sun. 2019. FIRM-AFL: High-Throughput Greybox Fuzzing of IoT Firmware via Augmented Process Emulation. In *28th USENIX Security Symposium (USENIX Security 19)*. 1099–1114. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/zheng.

A An Empirical Study

To investigate the risks of user-controllable inputs driving interface operations and their semantic associations, we conducted an empirical study. We defined user-controllable inputs that drive interface operations as core parameters.

We collected 477 reproducible reports from historical CVEs of mainstream vendors, including Netgear, D-Link, Linksys, Tenda, TOTOLINK, and TP-Link, to form our benchmark dataset.

Through reverse engineering, we verified that 80.92% (386 out of 477) of the triggering parameters in these vulnerabilities were core parameters. In contrast, within a control group of 477 non-vulnerable parameters randomly sampled from the same interfaces, only 28.30% (135) were core parameters. A chi-square test confirmed this difference is statistically significant ($\chi^2 = 266.4$, $p < 0.001$). These findings indicate that core parameters are significantly over-represented in vulnerabilities. This is because core parameters are more likely to flow into sensitive sinks in back-end programs, such as memory operations or command executions, and therefore present higher security risks.

To evaluate the feasibility of identifying core parameters using semantics in a black-box scenario, we designed a double-anonymous expert review. We extracted ⟨Interface URI, Triggering Parameter Name⟩ tuples from the CVE descriptions in the benchmark dataset. We then invited three experts, each with over five years of experience in IoT vulnerability research, to conduct the review. To ensure objectivity, the process employed an independent, double-anonymous mechanism in which experts were unaware of one another. Experts were required to determine whether the triggering parameter name semantically indicated that it was a core parameter for the interface's operation, based on the provided tuples and full datapacket. This was a binary classification task (Yes/No). Disagreements were resolved by majority vote.

Among the 386 verified core parameters, experts could directly identify 332 through naming semantics. Furthermore, experts correctly excluded 74 non-core parameters (out of 91). To ensure the objectivity of the semantic definitions, we calculated Cohen's Kappa (k=0.73), indicating substantial inter-rater agreement among the experts. Notably, experts were provided only a limited subset of observable information (URIs and datapackets) in the black-box setting. In practice, more information is actually available in real-world settings, such as device front-ends and client UI.

B Operation Logic Mapping Artifacts

B.1 Chosen Devices List to construct Database

Table 4 shows the 46 devices from 9 mainstream IoT vendors used to construct the Functional Categories and Operation Logic Database.

Table 4: Chosen Devices List to construct Functional Categories and Operation Logic Database

Vendor	Device	Type	Interaction	Vendor	Device	Type	Interaction
D-Link	AC3150 / DIR-X1560 / DSL-3782 / DWR-921	Router	web	Linksys	WRT1900AC / EA7500 / E5600	Router	web
	DCS-930L	Camera	web		WVC80N	Camera	web
	COVR-1102	Mesh / AP	web		RE7000	Extender	web
					WHW03 (Velop) / MR6350	Mesh / AP	app
					LAPAC1200	Mesh / AP	web
Netgear	R7000 / RAX40	Router	web	TOTOLINK	A3700R / LR350 / NR1800X	Router	web
	EX7300v2	Extender	web		X5000R	Router	web
	WNCE2001	Media Bridge	web		EX200	Extender	web
Cisco	RV340	Router	web	TP-Link	TL-WR1043N / AX10 / C7	Router	web
	WAP125	Mesh / AP	web		GE400	Router	web + app
Motorola	MG7310	Router	web		Omada 8MP Bullet	Camera	app
	MT7711	Router	web		VIGI C340	Camera	web + app
					RE500X / TL-WA860RE / TL-WPA7510 KIT	Extender	web
					Deco M4	Mesh / AP	app
Tenda	AC10 / AX3 / TX12 / N301 / 4G06	Router	web	Belkin	N600 DB / AC1800	Router	web
	A15	Extender	web		F9K1122v2	Extender	web
	O3	Media Bridge	web				

Note: The **Interaction** column records how a device's management interface is actually exposed. **41** are managed through a web interface, and **2** are managed through both a web interface and a mobile app (Archer GE400 and VIGI C340).

B.2 Functional Categories

We grouped 22 functional categories: WiFiSettings, ParentControl, VPN, USB, Bandwidth, Power, LED, Filter, Firewall, IPTV, Route, DNS, DMZ, UPnP, WAN, DHCP, LAN, Time, Login, RemoteControl, Diagnostic, and Logging.

First, we collected UI items and functional modules from these management interfaces and organized the interface operations into standard verb–object labels. We obtained this information from each device's web interface, mobile app, or manual playbook. Second, we used the cosine similarity between sentence embeddings to deduplicate redundant labels, merging every pair above 0.9 (single linkage). We also manually removed operations that exist only in a single device. Finally, we grouped the labels by the UI navigation subpage and the functional modules they belong to: labels reached from the same navigation subtree and acting on the same subsystem are grouped into the same functional category.

B.3 Prompts

Prompt 1: Category classification

I will provide information from a network device's management interface. Your task is to classify this information into the most appropriate category.

Input Data:
- UI Text/ID: {ui_info}
- Endpoint/URL Path: {uri}

Classification Rules:
1. Select the most relevant category from this list: {category_list}
2. Base your decision on the functionality implied by the UI information and URL.
3. If multiple categories seem relevant, choose the most specific one.

Output Requirement:
Return ONLY a JSON object: {{"category": "category_name"}}

Prompt 2: Operation identification

You are analyzing IoT device functionality to determine the specific operation logic label and operation type.

INPUT INFORMATION:
1. UI Information: {ui_info}
2. URI Path: {uri}
3. Candidate Functions (from database): {json.dumps(candidate_list, ensure_ascii=False, indent=2)}

TASK:
Using the input information above, you need to:
Select the most appropriate operation_logic_label from the candidate functions list. Only when no candidate is semantically relevant at all may you create a new one, following the format {prefix}<function_name> with <action>_<target> naming.

OUTPUT REQUIREMENTS AND FORMAT:
{{"operation_logic_label": "MUST be selected from candidates if any match exists. Only generate new if NO candidate matches."}}

C Experiment Results

The CVEs we found are listed in Table 5.

C.1 Execution Trace

Initialization. To clearly specify the recommended workflow for applying DetFuzz to a new device, we use the TOTOLINK A3700R as an example. Initialization requires two steps: (1) get the device IP address; since the A3700R requires login, log in manually once via the web client. (2) submit requests for each interface via the A3700R's web client and capture the requests with Burp Suite. Get the session cookie (SESSION ID, value 2:1767…) from Burp and replay it in the PoC. For this device, the automatic Selenium crawl is empty, so DetFuzz finished this pipeline without this information.

LLM interaction. DetFuzz makes four LLM calls per interface; this is an example of the setWiFiEasyGuestCfg interface:

• Functional Categories.

Table 5: Detailed CVE List Detected by DetFuzz

Func	Operation	Device	CVE
LAN	set_static_ip	1	CVE-2025-15218
		2	CVE-2025-5851
		3	CVE-2025-5609
WIFI	schedule_wifi_time	1, 3	CVE-2024-2710, 2490
	set_security_mode	8	End-of-Life*
	set_guest_ssid	11, 13	CVE-2026-1155, 1143
		8, 10	End-of-Life*
	set_wifi_name	11	CVE-2026-1156, 1157, 1158
		12, 13	CVE-2026-1328, 2024-42545
VPN	set_server_ip_range	1, 3	CVE-2024-2764, 2488
	set_user_list	1, 2, 3, 5	CVE-2025-15215, 5848, 5607, 15217
Diagnostic	ping_diagnosis	11	CVE-2026-1149
	traceroute_diagnosis	11, 12	CVE-2026-1150, 1327
WAN	set_op_mode_cfg	11, 12, 13	CVE-2024-7214, 2026-1326, 2024-7160
Firewall	set_firewall	1, 2, 3	CVE-2024-2704, 2809, 28547
DHCP	set_dhcp_reservation	5	CVE-2025-15216
Bandwidth	set_qos_band	1, 3	CVE-2024-2705, 2489
Remote Control	set_remote_web_config	2	CVE-2025-5849
Route	add_static_route	1	CVE-2024-2709

* **End-of-Life (EoL)**: Indicates vulnerabilities confirmed by the vendor (e.g., TP-Link) in legacy products that no longer receive security updates or formal CVE assignments.

Input: URI 'setWiFiEasyGuestCfg', UI item 'Guest Network ', and functional categories database.

Output: 'category = WiFiSettings'.

- Operation Logic labels.

Input: all Operation Logic labels within WiFi category.

Output: 'wifi.set guest ssid, operation type = set'.

- Generate inferred core parameter.

Input: URI 'setWiFiEasyGuestCfg' and Operation Logic label 'wifi.set guest ssid'.

Output: 'guestSsid'.

- Prerequisites Analysis.

Input: Because the front-end code is missing, the LLM uses only the packet parameters and prompt rules.

Output:

```
{"prerequisites":
  [["accessEnabled2g=1",
    "accessEnabled2g=0"],
   ["wifiOff=0","wifiOff=1"],
   ["accessEnabled5g=1",
    "accessEnabled5g=0"]],
 "other_param":
  [["ssid=TOTOLINK_..."],
   ["key="],["hssid=0"],["merge=0"],
   ["topicurl=setWiFiEasyGuestCfg"]]
}
```

Similarity Matching. Calculate the cosine similarity between 'guestSsid' and each packet parameter.

#	Parameter	Sim.
1	ssid	1.000
2	hssid	0.800
3	hssid5g	0.714
4	ssid5g	0.667
5	accessEnabled2g	0.628
6	key	0.627
7	accessEnabled5g	0.612

Verification . Take 'ssid=overflow' as an example. The executor replaces the target parameter value overflow with 2,000 A and fills auxiliary parameters based on prerequisite analysis. Other fields keep the ordinary value (Content-Length is calculated by a 1,644-byte form body). The executor sends a POST request to /cgi-bin/cstecgi.cgi and prints the returned status code or the error text. This test unit should be double-checked. First, replay the ordinary request and make sure the interface is OK; then send the request with parameters replaced. If the output shows a segmentation fault, it triggers. Each payload is attempted at most 5 times, with a 10-second pause after a timeout, and each test unit has a 300-second wall-clock limit. After 'ssid=overflow' succeeds, the same test units (same target parameter and different prerequisites) are skipped.